

Created by the Aerospace Cyber Defence Centre of Excellence (CEC)

Testing guide **WAC (Windows Artefacts Collector)**

Windows artefact collection software
for digital forensic investigation



SYSTEM
ARTEFACTS



USER
FILES



DIGITAL
EVIDENCE

A TOOL IN SUPPORT
OF DIGITAL
INVESTIGATION

WAC — Windows Artefacts Collector

[Testing guide](#)

Software version 1.3.1

Document edition September 2026

Publisher Aerospace Cyber Defence Centre of Excellence (CEC)
French Air and Space Force Academy, Air Base 701, Salon-de-Provence

This document explains how to run each of the WAC tests by hand, what each one proves and does not prove, and how to read a failure. It is meant for whoever changes the code or wants to check a version before using it. The durations and volumes quoted were measured on the test virtual machine and on a real collection; they are orders of magnitude.

Contents

1 Principles	2
1.1 What a WAC test must prove	2
1.2 Three rules	2
2 Setting up	3
2.1 The Linux workstation	3
2.2 Building WAC and the harnesses	3
2.3 The test virtual machine	4
3 The test cycle in the virtual machine	5
3.1 Running a cycle	5
3.2 Checking the outputs	5
3.3 Comparing two runs	6
4 The format harnesses	7
4.1 Fingerprints: sha_test	7
4.2 Decompression: lznt1_test and xpress_test	7
4.3 Event messages: wevt_test and event_messages_test	7
4.4 Event logs: evtx_test	8
4.5 Exhibit store: consign_test	8
4.6 Raw reading: raw_hive_test	8
4.7 Signatures: authenticocode_test	8
5 The robustness harnesses	10
5.1 The principle	10
5.2 Shortcuts: lnk_test	10
5.3 Jump lists and Prefetch: parsers_test	10
5.4 Reference	11
6 Cross-checking on a real collection	12
7 Reading a failure	13

7.1	Exit codes	13
7.2	Locating a read outside the buffer	13
7.3	An endless loop	13

Principles

1.1 What a WAC test must prove

WAC produces JSON files. That they are valid proves almost nothing: a field read at the wrong offset, a date interpreted in the wrong time zone, a name decoded with the wrong code page all give perfectly valid JSON — and wrong JSON. The costliest defects found in WAC were all of that kind: they showed only when a value was confronted with another source.

The tests therefore fall into three families, which do not answer the same question:

The virtual machine cycle (chapter 3): does WAC run to the end on a real Windows, and are its outputs consistent with each other? The *cross-checks* confront a value computed by WAC with an independent value from the same collection.

The format harnesses (chapter 4): does a decoder give exactly the result of the reference — Microsoft's own tool, a published standard, another implementation?

The robustness harnesses (chapter 5): does a parser read outside its buffer, or loop for ever, on a truncated or forged file?

1.2 Three rules

1. **A fix comes with the test that would have caught the defect.** And that test is checked both ways: it passes on the fixed output, and it fails when the defect is replayed.
2. **After any change, compare with the previous run,** not just the green marks: an artefact that went from 1,522 entries down to 573 lights no warning at all (section 3.3).
3. **Real data does not leave the test workstation.** A real collection holds personal data: it is processed locally, its temporary copies are deleted after use, and none of it enters the repository (`vmtest/results/` is ignored by git).

Setting up

2.1 The Linux workstation

Everything is built on Linux, cross-compiled with MinGW-w64; the harnesses run natively, under wine, or in the virtual machine.

```
sudo apt install g++-mingw-w64-x86-64 wine64 python3 python3-pip
pip install olefile          # independent OLE reader, for the cross-checks
```

2.2 Building WAC and the harnesses

```
./build-windows.sh --test
```

Output	Role
build-windows/WAC.exe	the collection tool
build-windows/raw_hive_test.exe	raw NTFS reader, used by the VM cycle
build-windows/tests/lnk_test.exe	robustness of the shortcut parser
build-windows/tests/parsers_test.exe	robustness of the jump list and Prefetch parsers
build-windows/tests/propsys.dll, offreg.dll	stand-ins for wine, which lacks both libraries

Warning

The two stand-ins are deliberately in build-windows/tests/, away from WAC.exe. Beside it, they would be loaded instead of the real libraries — Windows looks in the program's folder first — and WAC would read no hive at all, with no visible error. Never copy the tests/ folder onto a collection medium.

The build must end without a single warning: -Wall -Wextra are on, and warnings have already pointed at real defects.

The other harnesses (chapter 4) are each built with a command given in their header and repeated in this guide; several compile natively on Linux.

2.3 The test virtual machine

The VM is a Windows 11 driven through the *qemu-guest-agent*: no interaction, no network, commands run as NT AUTHORITY\SYSTEM. Its creation and prerequisites are described in `vmtest/README.md`; in short:

```
cd vmtest
ISO_WIN=~/Downloads/Win11_25H2_English_x64.iso ./create-vm.sh  # once
python3 qga.py ping                                         # does the agent answer?
python3 qga.py run --shell "dir C:\""                       # run a command
python3 qga.py write <host> <guest>                        # send a file
python3 qga.py read  <guest> <host>                        # fetch a file
```

Keep in mind

A command started with `qga.py run` is cut after 30 minutes. For a longer test (for instance the Prefetch files of a real machine, section 5.3), start it as a detached task in the VM and follow its log.

The test cycle in the virtual machine

3.1 Running a cycle

```
cd vmtest
./run-wac-test.sh --build      # rebuild, then full cycle
./run-wac-test.sh             # full cycle with the binaries already built
./run-wac-test.sh --raw-only  # only the validation of the raw reading
```

The cycle chains: upload of WAC.exe and raw_hive_test.exe into the VM, validation of the raw extraction of a hive (reg load must refuse it until it is repaired, then accept it), run of WAC, then fetch, into the folder vmtest/results/<timestamp>/, of:

- run.log — the console output of the collection, which ends with END, Time elapsed when WAC went to the end;
- raw.log — the trace of the NTFS reader;
- the JSON files of the artefacts;
- exhibits/ — the exhibit manifest, its seal and the list of the files really present in the exhibit store.

3.2 Checking the outputs

```
python3 check-json.py results/<timestamp>
```

Each line starts with a green tick (check passed), a red cross (defect) or a “skip” sign (check not applicable: the artefact is absent or empty). The exit code is 0 if no check failed. Beyond JSON validity and the shape of paths, the script confronts independent sources:

Cross-check	What it has already caught
Prefetch hash / suffix of its file name	65 wrong hashes out of 270 (hexadecimal without padding)
Prefetch executable name / start of the file name	a name read 8 bytes too early (“\x11” everywhere)
Prefetch \VOLUME{guid} paths resolved	20,066 paths out of 20,066 unresolved
X / XUtc pairs	double time-zone shift
no session earlier than the boot	date shifts, a boot time 3.5 s late

Cross-check	What it has already caught
ShimCache date / NTFS date in the manifest	a UTC date treated as local (-2 h)
DestList entries / jump list shortcuts	the first 15 entries of every list lost
machine name of the events / <code>OperatingSystem.json</code>	check of the BinXML decoding
record identifiers unique per log	a stale chunk read again
seal / real fingerprint of the manifest	a retouched manifest
files in the exhibit store / manifest content	209 exhibits added after sealing

The full list, with the history of each check, is in `vmtest/README.md`.

3.3 Comparing two runs

A green mark does not tell whether a piece of data disappeared. After any change, compare the new collection with the previous one: number of entries of each artefact, and values lost or gained. Two precautions:

- a collection changes by itself from one run to the next: timestamps, new events, updated Prefetch files, the fingerprint of \$MFT. Compare *paired* items — for instance a Prefetch by its path hash and its last run — leaving aside the dates of the extracted copy;
- a value that looks “lost” may just be an empty field no longer emitted, or a renamed key: normalise before concluding.

CHAPTER 4

The format harnesses

Each one confronts a WAC decoder with a reference. All are excluded from the WAC build by their `_test.cpp` suffix.

4.1 Fingerprints: `sha_test`

Confronts SHA-1 and SHA-256 with the published vectors of FIPS 180-4, and with lengths 55 to 128 bytes, which exercise the padding — the one place where an otherwise correct implementation goes wrong.

```
cd WAC && g++ -std=c++17 -I. sha.cpp sha_test.cpp -o /tmp/sha_test && /tmp/sha_test
```

4.2 Decompression: `lznt1_test` and `xpress_test`

Check the NTFS (LZNT1) and WOF (XPRESS Huffman) decompressors against *Microsoft's own compressor*: a known file is compressed in the VM by `RtlCompressBuffer`, unit by unit, and the harness decompresses each unit and compares it byte for byte with the original. A wrong decompressor usually decodes without any error: only the exhaustive comparison reveals it.

```
cd WAC
g++ -std=c++17 -I. lznt1.cpp lznt1_test.cpp -o /tmp/lznt1_test
g++ -std=c++17 -I. xpress.cpp xpress_test.cpp -o /tmp/xpress_test
/tmp/lznt1_test <original> <compressed.bin> <index.idx>
/tmp/xpress_test <original> <compressed.bin> <index.idx>
```

`index.idx` holds one line `<n> <original size> <compressed size>` per unit; a null compressed size marks a unit Windows gave up compressing.

4.3 Event messages: `wevt_test` and `event_messages_test`

`wevt_test` checks the chain that turns an event into a sentence, on the files of a real provider: its DLL and its localised satellite.

```
cd WAC && g++ -std=c++17 -I. pe_resource.cpp wevt.cpp wevt_test.cpp -o /tmp/wevt_test
/tmp/wevt_test gpsvc.dll fr-FR/gpsvc.dll.mui "{aea1b4fa-97d1-45f2-a64c-4d69fffd92c9}" 1002
4001
```

event_messages_test does the same starting from the SOFTWARE hive; it runs on the examined machine or in the VM, the only places where the hive and the providers' binaries are readable:

```
event_messages_test <SOFTWARE hive> <guid> [id[:version] ...]
```

4.4 Event logs: evt_x_test

Reads an .evt_x outside Windows, under wine. It serves to try the decoder on logs the VM does not produce: old, forwarded, closed abruptly, damaged — and to compare record by record with an independent implementation (python-evt_x).

```
wine evt_x_test.exe "Z:/path/Security.evt_x"          # summary per log
wine evt_x_test.exe "Z:/path/Security.evt_x" 3        # + XML of 3 records
wine evt_x_test.exe "Z:/path/Security.evt_x" --dump   # one record per line
wine evt_x_test.exe --collect "Z:/tmp/tree" "Z:/tmp/out"      # complete chain
wine evt_x_test.exe --collect-memory "Z:/tmp/tree" "Z:/tmp/out2" # same chain, in memory
cmp /tmp/out/events.json /tmp/out2/events.json          # must be silent
```

The build command is in the README, section *Testing the EVT_X parser outside Windows*.

4.5 Exhibit store: consigne_test

Replays the production chain — fingerprints, manifest, verified copy, hive replay — on hives given as arguments, and checks what no build reveals: the exhibit store stays identical byte for byte while the working copy is changed, and the seal carries the manifest's real fingerprint.

```
wine consigne_test.exe "Z:/tmp/out" "Z:/path/SYSTEM" "Z:/path/ntuser.dat"
```

4.6 Raw reading: raw_hive_test

Extracts a file by reading the NTFS volume directly. To be run on Windows, as an administrator; the VM cycle uses it to validate the extraction of a hive.

```
raw_hive_test.exe [volume] [path] [output] [--fix] [--list] [--attrs]
```

--fix makes the extracted hive loadable, --list lists a directory, --attrs shows the \$MFT attributes of the file without interpreting them.

4.7 Signatures: authenticode_test

Verifies the catalogs of a folder, then gives a verdict per file; its verdicts are compared with those of Get-AuthenticodeSignature on the same files, in the VM. The only gap that matters:

a file WAC accepts and Windows refuses.

```
authenticode_test --catalogs <folder>  
authenticode_test --catalogs <folder> --files <list>
```

The robustness harnesses

5.1 The principle

A parser reads at offsets the file itself declares. On a truncated or forged file, a read past the buffer does not crash on its own: it reads the neighbouring memory and publishes it, in valid JSON. The harnesses make it crash: each input is copied against a forbidden memory page (PAGE_NOACCESS), once *ending* against it (reads past the end), once *starting* right after it (reads before the start, for instance at a negative offset taken from the data). A single byte read outside the input raises an access violation.

For each file given, the harness parses:

- the whole file;
- its truncations (every length for small files);
- copies with bytes overwritten at random, with a fixed seed: a failure reproduces identically.

5.2 Shortcuts: lnk_test

```
cd build-windows/tests
WINEDLLOVERRIDES="propsys=n" wine lnk_test.exe a.lnk b.lnk ...
```

Where to find files: the Recent folder and the Start menu of the VM, or those of a collection (Users\<name>\AppData\Roaming\Microsoft\Windows\Recent). Accented or non-Latin names travel badly through the wine command line: copy the files under numbered names.

Keep in mind

WINEDLLOVERRIDES="propsys=n" loads the propsys.dll stand-in: wine does not implement the function that names properties, and would stop the test at the first real shortcut carrying some.

5.3 Jump lists and Prefetch: parsers_test

```
parsers_test.exe jumplist-auto <file.automaticDestinations-ms> ...
parsers_test.exe jumplist-custom <file.customDestinations-ms> ...
parsers_test.exe prefetch <file.pf> ...
```

Jump lists are tested under wine, like `lnk_test`. Prefetch files are tested **in the VM**: they are compressed (XPRESS Huffman), and only the `ntdll` library of Windows decompresses them. The harness decompresses each file before trying it: it is WAC's parser it aims at, not Windows' decompressor.

For the 307 Prefetch files of a real machine (over a million inputs), count about two hours in the VM: run the test as a detached task, in batches, and follow its log. Example PowerShell script, put into the VM with `qga.py write`:

```
$files = Get-ChildItem C:\wactest\pt\pf\*.pf | ForEach-Object { $_.FullName }
$log = 'C:\wactest\pt\pf_results.txt'
for ($i = 0; $i -lt $files.Count; $i += 20) {
    $batch = $files[$i..([Math]::Min($i + 19, $files.Count - 1))]
    Add-Content $log (& C:\wactest\pt\parsers_test.exe prefetch @batch 2>&1)
    Add-Content $log ("batch " + $i + " exit=" + $LASTEXITCODE)
}
Add-Content $log 'DONE'
```

5.4 Reference

Results obtained on a real machine, each input parsed on both sides of the forbidden page:

Files	Inputs	Where
220 shortcuts	546,214	wine
48 automatic jump lists	175,585	wine
23 custom jump lists	79,036	wine
307 Prefetch files	1,246,491	VM, about 2 h

Cross-checking on a real collection

A freshly installed VM produces no fragmentation, no old jump lists, no exotic file names. Several defects only showed on a real collection, confronted with an independent source:

- **jump lists / olefile:** the streams and the DestList, read again by an independent OLE reader, must give exactly the same entries (numbers, dates, paths). That is what showed the streams are named "1" to "f", not "01" to "0f";
- **shortcut / DestList:** in a jump list, the target of a shortcut and the path of its DestList entry are two records of the same file. Their disagreement revealed a wrong code page ("d\x92orientation") and an ignored Unicode path;
- **events / python-evtx:** record-by-record comparison (evtx_test --dump);
- **signatures / Get-AuthenticodeSignature** (authenticode_test);
- **old collection / new one:** decode the same files with the current code and compare, after normalisation, with the output of a previous version.

Warning

A real collection is processed locally. If a hive must be read with the real `offreg.dll`, it may be copied into the test VM, then deleted as soon as the test is over. No copy stays in the temporary working folder, and none of it enters the repository.

Reading a failure

7.1 Exit codes

Code	Meaning
0	everything passed
1	a check failed, or an input file is unreadable
2	wrong usage (arguments)
3	parsers_test: a parse did not end within 60 s (watchdog)
0xC0000005 (-1073741819)	access violation: a read outside the buffer

7.2 Locating a read outside the buffer

Under wine, a fault prints the call stack, with addresses relative to the program (lnk_test+0x485bb). On Windows, parsers_test itself prints the input at fault and the faulting address:

```
FAILED exception 0xC0000005 at exe+0x78b70, address read 0x159deb4200c:
C:\wactest\pt\pf\CMD.EXE-0BD30981.pf truncated at 192 (guard after)
```

The address is translated into a function with the program's symbol table (base address 0x140000000):

```
x86_64-w64-mingw32-nm -C parsers_test.exe | grep -i " t " | sort > syms.txt
# look for the symbol whose address is just below 0x140000000 + 0x78b70
x86_64-w64-mingw32-objdump -d --start-address=0x140078b40 \
--stop-address=0x140078b80 parsers_test.exe
```

The disassembled instruction gives the offset read (here 0xcc(%r14)); it is then matched, in the source code, with the field read at that position. Then replay the exact input (the truncation or corruption shown) to check the fix, and the whole series after it.

7.3 An endless loop

If parsers_test stops on “no end after 60 s”, the input named keeps a parser running for ever — typically a cyclic sector chain in an OLE container. It is a defect just like a read outside

the buffer: a forged file would be enough to block a collection.